

FeedbackHub

What the app must do — written in simple words

THIS FILE	The full study of the job. Every case, every rule, every screen, the data, and the pictures.
WHERE IT COMES FROM	references/FeedbackHub-Assignment.pdf. If this file and the brief do not agree, the brief wins.
STATE	Nothing is built yet. This is a plan of what must be true, not a report of what is done.
WORDS USED	MUST the brief asks for it · GOOD makes it better, can be cut · LATER not now
NUMBERS	Every rule has a number (R-1, R-2, ...) so we can point at it later.

WHAT IS INSIDE

- 1 What the app is
- 2 The hard parts (read this first)
- 3 Who uses it
- 4 Who can do what
- 5 Use case picture
- 6 The journeys from the brief
- 7 The screens
- 8 Rules, feature by feature
- 9 How a request moves (picture)
- 10 How the app starts (picture)
- 11 Settings and where they live (picture)
- 12 The data
- 13 The ERD
- 14 Rules that are always true
- 15 Every case on every screen
- 16 Things that are not features
- 17 Tools we must choose
- 18 How we know a feature is done
- 19 What is not clear in the brief
- 20 Where each rule comes from

1. What the app is

FeedbackHub is a feedback board for people inside one company.

- People write feature requests and feedback.
- Everyone can read them, search them, and vote for the ones they like.
- People talk about a request in comments.
- An admin sets the status of a request, keeps the category list tidy, removes bad comments, and changes the settings of the app.

Why it exists, in the words of the brief: **stop the same idea coming five times by email**, and **show what is really being worked on**.

Two things follow from that one sentence, and we treat them as real work, not extras:

1. Finding an old request must be easy. So: search, filters, and a hint at writing time that shows requests that look the same.
2. The status must be easy to see everywhere a request is shown.

SIZE WE PLAN FOR

One company. One board. Around a few hundred people, a few thousand requests, a few tens of thousands of votes. Every speed number in part 16 is for this size.

2. The hard parts (read this first)

Some parts of this job are easy to get wrong, and the brief says they carry the most marks. They are listed here so they are not forgotten.

#	THE HARD PART	WHY IT IS EASY TO GET WRONG
H-1	The AI write-up	The brief says it carries the most weight. It cannot be written at the end from memory. The prompts and the mistakes must be saved while working.
H-2	Checks on the server, not in the screen	Hiding a button is not a check. If someone calls the server directly, the server must still say no. This is the easiest way to fail on security.
H-3	Loading, empty and error states	The brief says these are part of the work, not polish for later. They are always the first thing dropped when time is short.
H-4	One call at start, not a chain	The brief asks for this by name. The easy way is four or five calls one after the other, and that is the wrong answer.
H-5	A feature switch that really changes the app	Hiding a button is not a feature switch. The server must refuse the action too.
H-6	Categories and statuses are data	If they are fixed in the code, the admin cannot change them, and the brief says the admin must.
H-7	One vote per person	Easy to say, easy to break with a double click. The database must stop it, not the code.
H-8	One command starts everything, with data	It works on my computer and fails on the reviewer's. It must be tested from a fresh copy.
H-9	The same style everywhere	When code is written in bursts, the same job ends up done three different ways.
H-10	Saying what does not work	Cheap to do, and the brief gives marks for it.

What this means for the order of work: H-2, H-3 and H-9 all say the same thing — build the shared parts first (the permission check, the error shape, the loading and empty and error boxes), then build features on top of them.

3. Who uses it

WHO	TYPE	WHAT THEY ARE
Visitor	Person	Not signed in. Can only see the sign-in page. (See question Q-3 in part 19.)
User	Person	A signed-in person from the company. Writes, reads, votes, comments. Owns their own content and their own settings.

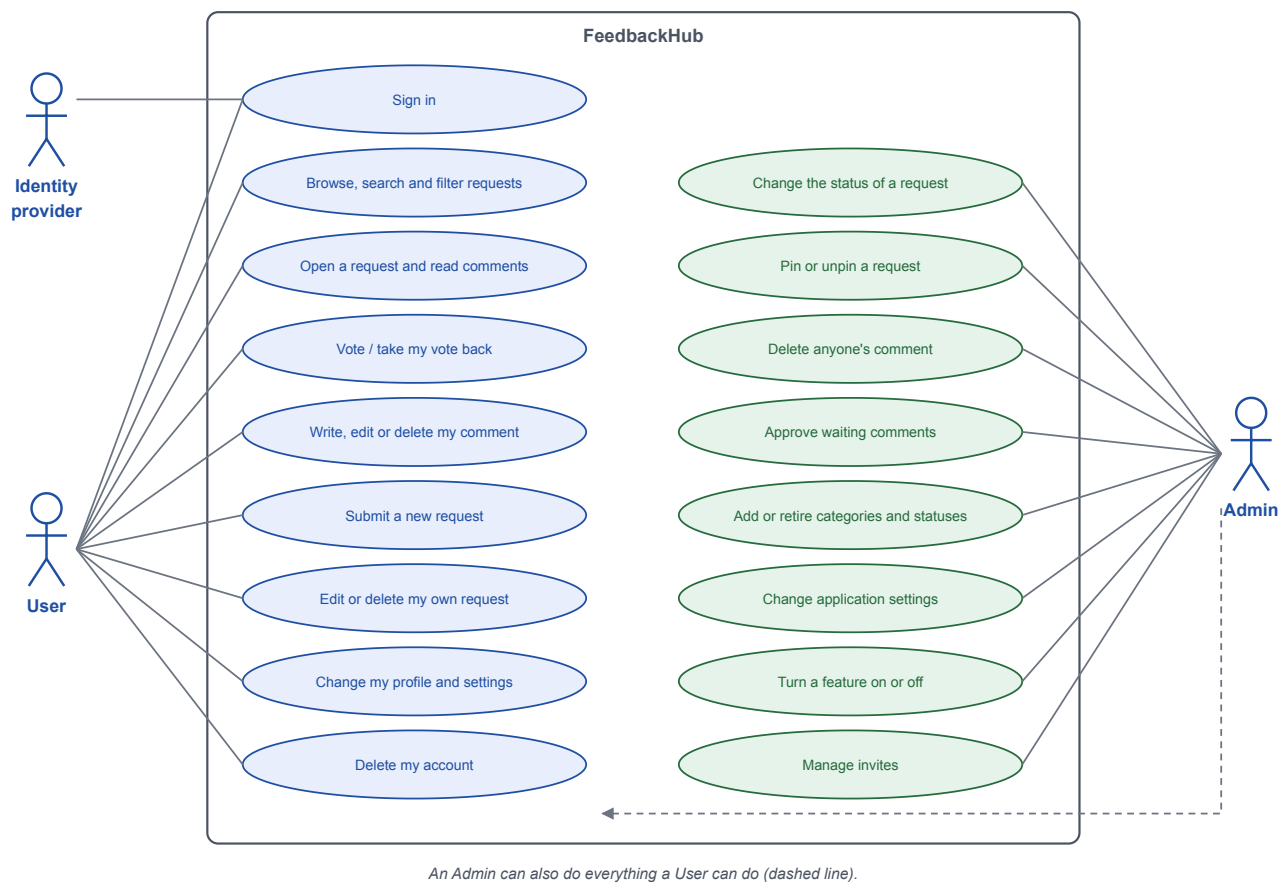
Admin	Person	Someone from the product team. Can do everything a User can, plus set statuses, tidy the categories, remove bad comments, and change app settings.
Identity provider	System	An outside open-source system. It owns the passwords, the sign-in page, the sign-up page, and social sign-in. We never write these ourselves.
Mail sender	System	The part that sends the emails people asked for. In this version it only writes them down; it does not send them. Part 19 says so plainly.

4. Who can do what

This table is a rule, not a description. Every line is checked on the server. Every **no** gets a test that proves the server says no.

ACTION	VISITOR	USER	ADMIN
See the board and open a request	no	yes	yes
Write a new request	no	yes, up to the limit	yes
Edit a request (title, text, category)	no	only mine	any
Delete a request	no	only mine	any
Change the status of a request	no	no	yes
Pin or unpin a request	no	no	yes
Vote, or take my vote back	no	yes, even on my own request	yes
Write a comment	no	yes, if comments are switched on	same
Edit a comment	no	only mine	only mine. An admin removes a bad comment. An admin never rewrites what someone said.
Delete a comment	no	only mine	any
Approve a waiting comment	no	no	yes
Add or retire a category or status	no	no	yes
Change app settings and feature switches	no	no	yes
Manage invites	no	no	yes
Change my own profile and settings	no	yes	yes
Delete my own account	no	yes	yes, unless last admin (R-62)
See another person's email or settings	no	no	no — nothing in the brief needs it

5. Use case picture



Picture 1. Everything a person can do. Blue = things a User does. Green = extra things only an Admin does. The Identity provider is outside our app: it owns sign-in.

6. The journeys from the brief

The brief lists eleven journeys. Every one is copied here, cut into steps, with the rules that make each step work. Nothing from the brief is dropped.

6.1 User journeys

#	FROM THE BRIEF	WHAT MUST HAPPEN	RULES
U-1	Registers or signs in through the identity provider.	The person is sent to the identity provider. They sign in with email and password, or with a social account. They come back signed in. On the first time only, we make a local record for them — but only if the sign-up rule allows it.	R-1...R-9
U-2	Lands on the request list: sorted, filterable by status and category, searchable by text, paginated.	The board opens. The person can sort it, filter by status, filter by category, type words to search, and move page by page. Their own saved sort and filters are used the first time.	R-16...R-25

U-3	Opens a request, reads the discussion, upvotes it, adds a comment.	The request page shows the text, who wrote it, the status, the votes, and all comments. The vote button works with one click and with the keyboard. The comment box adds a comment without reloading the page.	R-13, R-26...R-31, R-32...R-42
U-4	Submits a new request; sees it appear in the list.	A form with title, text and category. The status is set by the server. After saving, the person goes to their new request and it is on the board.	R-10...R-15
U-5	Finds their own earlier request, edits its description, and later deletes it.	The person can find their own requests. They can change the text. Later they can delete it, after a clear question that says what will be lost.	R-13, R-14, R-15
U-6	Updates their profile and application preferences.	A settings screen: name, picture or initials, theme, language, default sort, default filters, email choices, and delete my account.	R-54...R-62

6.2 Admin journeys

#	FROM THE BRIEF	WHAT MUST HAPPEN	RULES
A-1	Signs in and reviews recently submitted requests.	The admin opens the board sorted newest first, filtered to the first status. This is one saved link, not a new screen.	R-63
A-2	Changes a request's status and pins an important one to the top of the list.	The admin can change the status from the board and from the request page. The change is saved with who did it and when. A pinned request goes to the top.	R-64, R-65, R-23
A-3	Deletes an inappropriate comment.	The comment is replaced by a small grey line saying it was deleted. The thread does not break. We save which admin did it.	R-38, R-66
A-4	Adds a new category and retires an unused one.	The admin can add, rename, recolour and reorder. Retire means hide from the picker — old requests still show it correctly. Deleting one that is in use is refused.	R-43...R-50
A-5	Adjusts application-wide settings.	Sign-up rule, comment approval on or off, how many requests a person may send, feature switches, and the default sort. It works right away, with no restart.	R-51...R-53, R-67...R-70

7. The screens

SCREEN	WHO	WHAT IS ON IT
Sign in	Visitor	Owned by the identity provider. We only send the person there and take them back.
Board (home)	User, Admin	Search box, status filter, category filter, sort, the list, page buttons. Pinned requests at the top. An admin can also change the status from here.

Request page	User, Admin	Title, text, category, status, who wrote it, when, vote button, the comment list, the comment box. Status history. Edit and delete for the owner or an admin.
New request	User, Admin	Title, text, category. Shows similar requests while typing the title.
Edit request	Owner, Admin	Same fields as new. Not the status.
My settings	User, Admin	Name, picture, theme, language, default sort, default filters, email choices, delete my account.
Admin: categories and statuses	Admin	Add, rename, colour, reorder, retire. Shows how many requests use each one.
Admin: app settings	Admin	Sign-up rule, allowed email domains, comment approval, request limit, feature switches, default sort.
Admin: waiting comments	Admin	Only when comment approval is on. Approve or reject.
Admin: invites	Admin	Only when the sign-up rule is invite-only. Add an email, see if it was used.
Not allowed / Not found	All	Clear message, and a way back.

8. Rules, feature by feature

Every rule has a number. “The server” means the back end. When a rule says **the server**, it means the check happens there and hiding it in the screen is not enough.

8.1 Sign in and accounts

#		RULE
R-1	MUST	All sign-in is done by the outside identity provider. We never store passwords, never hash them, never write a password-reset flow, never write the social sign-in dance.
R-2	MUST	A person can sign in with email and password , or with at least one social account . Both screens belong to the identity provider.
R-3	MUST	The browser gets its token in the safe way (code + PKCE). No secret key is kept in the front end. The token is kept in memory, not in browser storage.
R-4	MUST	The first time a person signs in, the server makes a local record for them — but only after checking the sign-up rule (R-51). If the rule says no, they get a clear message and no record is made .
R-5	MUST	The server checks every token: the signature, who made it, who it is for, and that it is not old. It never trusts a token it did not check.
R-6	MUST	Every part of the app needs a signed-in person, except the health check and the sign-in page. Not signed in → 401 . Signed in but not allowed → 403 . These two are never mixed up.
R-7	MUST	The server decides “is this allowed” by reading the saved row from the database — never by trusting an id or a role sent in the request.
R-8	MUST	Being an admin is read from the checked token and compared with the local record before any admin action.
R-9	MUST	Sign out clears the app and ends the session at the identity provider too.

8.2 Writing and changing a request

#		RULE
R-10	MUST	A request has a title , a description and a category . The person picks these three. The server sets the status, the author, the time, and the counts.
R-11	MUST	The server gives every new request the status marked as the first one. A person can never choose or send a status.
R-12	MUST	Checks, on the server and in the form: title 5 to 120 letters, description 10 to 5000 letters, the category must exist and be active. A bad value gets a message under the field that says how to fix it.
R-13	MUST	The person who wrote it can change the title, text and category. An admin can change any request. After a change the page shows “edited”.
R-14	MUST	The person who wrote it can delete it. An admin can delete any. Deleting removes its votes and its comments too. We ask first, and the question says what will be lost.

R-15	GOOD	While the person types a title, we show up to five requests that look the same. It is only a hint. They can still write theirs. This is the whole point of the product.
------	------	---

8.3 The board

#		RULE
R-16	MUST	The board shows: title, category, status, who wrote it, number of votes, number of comments, a pin mark, and the last time something happened.
R-17	MUST	Search by words, over the title and the description . Not over comments.
R-18	MUST	Filter by status and by category. More than one can be picked in each. Inside one filter it means “or”; between the two filters it means “and”.
R-19	MUST	Sort by: newest, oldest, most votes, most comments, last activity.
R-20	MUST	The server only accepts sort names from a fixed list. Anything else is refused. A name from the user never goes into the database query as it is.
R-21	MUST	Pages. 20 per page by default, 100 at the most. The server never gives more, even if asked. The answer says how many there are in total.
R-22	MUST	The search, filters, sort and page are in the web address. So a board view can be copied, shared and found again with the back button.
R-23	MUST	Pinned requests come first — but only inside the filter the person chose. Pinning must never show a request the filter says to hide, and never hide one the filter says to show.
R-24	MUST	On the first visit the board uses the person's saved sort and filters. If the web address says something else, the web address wins.
R-25	MUST	Four different states: loading , nothing here yet , nothing matches your filters (with a Clear button), and something went wrong (with a Retry button).

8.4 Votes

#		RULE
R-26	MUST	One person can vote for one request one time , and can take the vote back. The database stops a second vote, not the code.
R-27	MUST	Voting twice, or un-voting when there is no vote, is fine. It gives back the current state instead of an error. A double click can never make two votes.
R-28	MUST	The number of votes is counted by the server from the real votes. The browser can never send a number.
R-29	MUST	A person may vote for their own request. A new request starts at zero votes, not one.
R-30	GOOD	The number changes at once when clicked. If the server says no, it goes back and shows why.
R-31	MUST	The vote control is a real button: it works with the keyboard, its name says the count and whether you voted, and the change is read out by a screen reader.

8.5 Comments

#		RULE
---	--	------

R-32	MUST	Anyone signed in can comment on any request. 1 to 2000 letters.
R-33	MUST	Comments are a flat list, oldest first, with pages. No replies inside replies.
R-34	MUST	Each comment shows who wrote it and when.
R-35	MUST	The writer can edit their own comment. It is then marked as edited.
R-36	MUST	An admin cannot edit someone else's comment. Moderation means deleting, never changing words.
R-37	MUST	The writer can delete their own comment. An admin can delete any.
R-38	MUST	A deleted comment leaves a small grey line: "This comment was deleted." The row stays so the thread still makes sense. The text is gone for good.
R-39	MUST	Deleted comments are not counted and cannot be edited.
R-40	MUST	When the admin turns on comments need approval , a new comment waits. Only the writer (marked "waiting") and admins can see it. It is not counted until it is approved.
R-41	MUST	An admin can approve a waiting comment (it appears) or reject it (it becomes a deleted line).
R-42	MUST	When the comments feature switch is off: the comment box and the thread are gone, comment counts are gone from the board, and the server refuses to save a comment with a clear message. This is the switch that proves H-5.

8.6 Categories and statuses

#		RULE
R-43	MUST	Categories and statuses are rows in the database , not fixed words in the code. The admin changes them while the app runs.
R-44	MUST	The admin can add, rename, recolour and reorder them. Two cannot have the same name, even with different capital letters.
R-45	MUST	Retire means: gone from the picker, but still shown correctly on old requests that use it.
R-46	MUST	A category or status that is used by any request cannot be deleted . The database refuses it. Only retiring is possible.
R-47	MUST	Exactly one status is marked as the first one. Marking a new one un-marks the old one, in the same step.
R-48	MUST	The first status cannot be retired. The last active category cannot be retired. Otherwise nobody could write a request.
R-49	MUST	The list of categories and statuses comes with the one start-up call (R-52). It is not a separate call.
R-50	GOOD	A status can be marked as an end status (like Done or Declined), so closed requests can look quieter and be hidden with one click. The admin screen shows how many requests use each value.

8.7 Where the settings live

The brief asks this by name: *where configuration lives, how it is resolved between global defaults and user overrides, and how the frontend obtains it without a chain of blocking requests on startup*. This is hard part **H-4**. Picture 4 shows the answer.

#		RULE
R-51	MUST	There are three places, and one order: code default , then admin setting , then user setting . The last one that is filled in wins. This is written one time , in one small function, and tested.
R-52	MUST	The browser gets everything it needs to start — who I am, my settings, the feature switches, the categories, the statuses — in one call , made one time when the app starts. A chain of calls is a bug.
R-53	MUST	Server addresses, database address and secrets come from the environment. They are never used to decide how the product behaves. Product settings never come from the environment.

8.8 My settings

#		RULE
R-54	MUST	A person can set their display name and a picture . With no picture we draw their initials. No file upload.
R-55	MUST	A person can set the theme : light, dark, or follow the computer.
R-56	MUST	The theme changes at once, and is right on the next load with no white flash. “Follow the computer” changes live when the computer changes.
R-57	MUST	A person can set the language . It changes at once, with no rebuild. A missing word shows in English, never as a code word.
R-58	MUST	A person can set their default sort and default filters for the board.
R-59	MUST	A person can choose their email messages : when someone comments on my request, and when the status of my request changes.
R-60	MUST	A person can only change settings from a fixed list of their own look-and-feel settings. Trying to change an admin setting is refused with a message, not quietly ignored.
R-61	MUST	A person can delete their account . Their name, picture and email are wiped, their sign-in stops working, their votes go. Their requests and comments stay, shown as “Deleted user”. The question we ask says this before they press the button.
R-62	MUST	The last admin cannot delete their own account. The app must never be left with nobody who can run it.

8.9 Admin jobs

#		RULE
R-63	MUST	An admin can see new requests: the board, newest first, filtered to the first status. One saved link is enough.
R-64	MUST	Only an admin changes a status. It can be done from the board and from the request page, and it shows at once.
R-65	MUST	Only an admin pins or unpins. More than one request can be pinned.
R-66	GOOD	Every status change is saved: the old status, the new status, which admin, and when. The request page shows this history.
R-67	MUST	The admin sets the sign-up rule : open to everybody / invite only / only these email domains. With a list of domains.
R-68	MUST	The admin can turn comments need approval on and off.

R-69	MUST	The admin sets a limit : how many requests one person may send, and in how long. The server counts and refuses number N+1.
R-70	MUST	Admin screens are hidden from normal people and refused by the server if someone types the address by hand. Every setting is checked when saved, and works with no restart. Deleting things always asks first and names the thing.

8.10 Emails

#		RULE
R-71	MUST	When something happens, we look at what the person chose, and write the message down in a list. Nobody is told about their own action.
R-72	MUST	Writing that message can never break or slow down the thing the person just did. A failed email must never undo a comment.
R-73	LATER	Really sending the email. Not in this version. We write down what we would have sent. The README says this plainly.
R-74	MUST	The list is built so that sending for real later is only a small change, not a rewrite.

8.11 The API, and running it

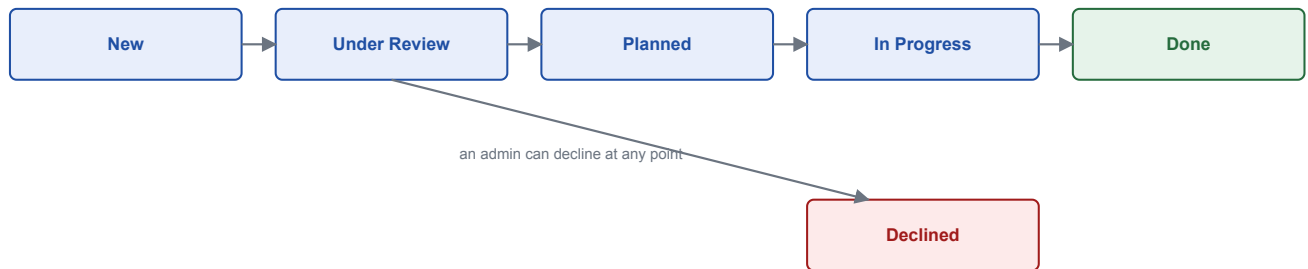
#		RULE
R-75	MUST	One web API with a version in the address. Normal status codes.
R-76	MUST	One shape for every error : a code we can read in the app, a message a person can read, the field names when a form is wrong, and an id we can search in the logs.
R-77	MUST	The API sends only the fields it means to send. We never send a database row straight out, or a new column leaks the day it is added.
R-78	LATER	An API document made from the code. The brief says it is optional. It is cheap, so we will do it.
R-79	MUST	Every part runs in a container: small, built in steps, running as a normal user, not as root.
R-80	MUST	One written command starts everything — front end, API, database, identity provider — with example data , from a fresh copy of the repository.
R-81	MUST	Files for a small local Kubernetes: how to run each part, how to reach it, settings and secrets kept apart, health checks, and limits.
R-82	MUST	Database changes run as their own step that finishes before the app starts. Never while the app is starting, because two copies would fight.
R-83	MUST	Two health checks: “is it alive” and “is it ready”. Ready means the database and the identity provider can be reached.
R-84	MUST	The same container image runs everywhere. Only the environment values change.

8.12 How it must feel to use

The brief says user experience is **part of the work, not polish for later**. So these are rules with tests, not wishes.

#		RULE
R-85	MUST	Every screen that loads data has: loading , empty , error , and where it makes sense not allowed . They are built one time and used everywhere, not made again on each screen.
R-86	MUST	While loading, the page keeps its shape (grey blocks the same size as the real thing). Nothing jumps when the data comes.
R-87	MUST	An error says what failed, whether trying again helps, and gives the button to try again. Never a raw error code or a stack trace.
R-88	MUST	A form message says how to fix it ("Title must be at least 5 letters"), sits next to the field, and shows when leaving the field or on save — not on every key press.
R-89	MUST	Everything works with the keyboard alone, in a sensible order, and you can always see where you are. A pop-up keeps the keyboard inside it and gives it back when it closes.
R-90	MUST	The layout works from a 360 pixel phone screen up. The page never scrolls sideways.
R-91	MUST	Anything that deletes asks first, names the thing, and says what will be lost. Nothing that cannot be undone happens on one click.
R-92	GOOD	Things that finish in the background (vote saved, comment added, settings saved, an error) are read out for screen readers.

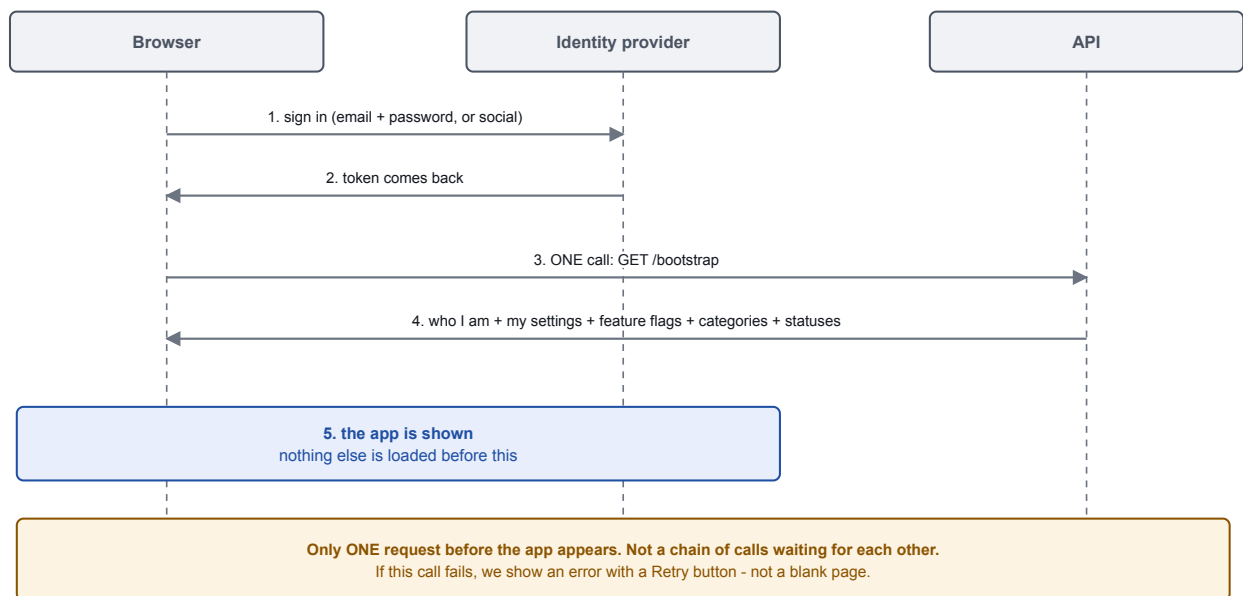
9. How a request moves



This is the normal path. An admin can move a request to any status. Statuses are data, so an admin can add or retire them.

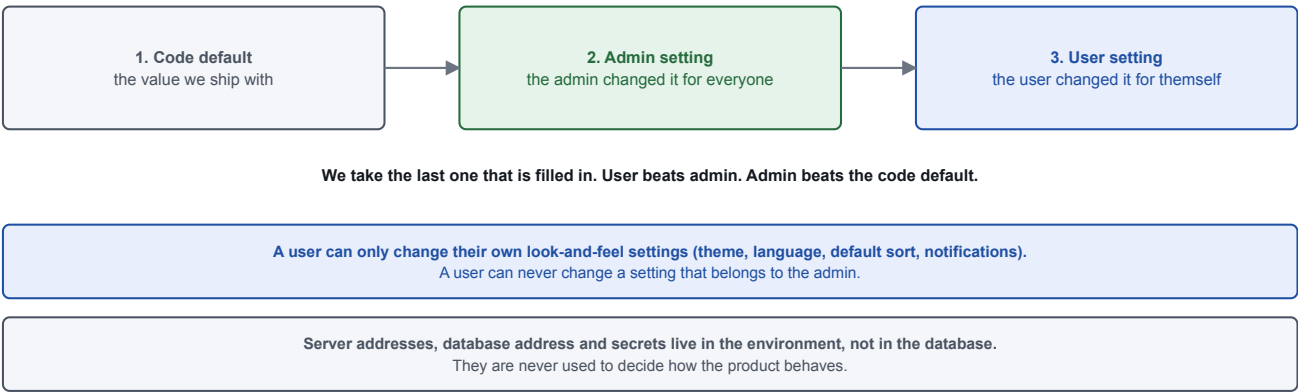
Picture 2. The normal path of a request. Only an admin moves it. Because statuses are rows in the database (R-43), an admin can add new ones or retire old ones, and this picture changes with them.

10. How the app starts



Picture 3. Sign in, then **one** call, then the app is shown. This is rule R-52 and hard part H-4.

11. Settings and where they live



Picture 4. How we pick the value of a setting (R-51), what a person is allowed to change (R-60), and what stays in the environment (R-53).

WHICH SETTING LIVES WHERE

WHERE	WHO CHANGES IT	EXAMPLES
Environment	The person who runs the servers	Database address, identity provider address, allowed web addresses, log level, secrets.
Admin settings (one row in the database)	Admin, while the app runs	Sign-up rule, allowed email domains, comments need approval, request limit and time window, feature switches, the default sort for everyone.
My settings (one row per person)	The person	Theme, language, default sort, default filters, email choices.

FEATURE SWITCHES

SWITCH	WHAT HAPPENS WHEN THE ADMIN TURNS IT OFF
comments	The important one. The comment box and the whole thread disappear. Comment counts disappear from the board. And the server refuses to save a comment, with a message saying comments are off. Screen and server (R-42).
pinned section	Pinned requests are shown as their own block at the top instead of inside the list.

A switch that only hides a button is **not** a feature switch. If the server still accepts the action, the switch does nothing. This is hard part H-5.

12. The data

Eleven tables. Every table has an `id`. Every date is kept in UTC. **PK** means the key of the row. **FK** means it points at another table. **UQ** means no two rows can be the same.

12.1 users

A copy of the person we get from the identity provider. **We keep no password here.**

FIELD	KIND	RULES	WHAT IT IS
<code>id</code>	<code>id</code>	PK	Our own id. All other tables point at this.
<code>external_id</code>	<code>text</code>	needed, UQ	The id the identity provider gave. This is how we know it is the same person.
<code>email</code>	<code>text</code>	needed, UQ	Used for the domain rule and for emails. Never shown to other people.
<code>email_verified</code>	<code>yes/no</code>	needed	Only a checked email counts for the domain rule (R-67).
<code>display_name</code>	<code>text</code>	needed, 1–80	Shown on requests and comments.
<code>avatar_url</code>	<code>text</code>	can be empty	Empty means we draw initials.
<code>role</code>	<code>text</code>	user or admin	Checked again on every admin action (R-8).
<code>status</code>	<code>text</code>	active or deleted	“deleted” means the account was wiped (R-61).
<code>created_at</code> , <code>updated_at</code> , <code>deleted_at</code>	<code>date</code>		When it was made, changed, wiped.

12.2 user_settings

One row per person, holding only what they changed. An empty field means “I did not change this, use the admin value”.

FIELD	KIND	RULES	WHAT IT IS
<code>user_id</code>	<code>id</code>	PK, FK → users	One row per person at most. Goes away with the person.
<code>theme</code>	<code>text</code>	light, dark, system	Empty = use the admin value.
<code>language</code>	<code>text</code>	can be empty	Same.
<code>default_sort</code>	<code>text</code>	from the fixed list	Same list as R-20.

default_status_ids	list of ids	can be empty	Board filter. A retired value is just skipped when reading.
default_category_ids	list of ids	can be empty	Same.
notify_on_comment	yes/no	can be empty	Comments on my requests.
notify_on_status_change	yes/no	can be empty	Status changes on my requests.

Why empty fields and not one big blob: empty says clearly “not changed”, every field keeps its own rule in the database, and picking the value stays one simple line (R-51). A blob would have no rules and no types.

12.3 app_settings — one row only

FIELD	KIND	RULES	WHAT IT IS
id	number	PK, always 1	Only one row can ever exist.
registration_policy	text	open / invite_only / domain_restricted	R-67.
allowed_email_domains	list of text	not empty for the domain rule	Kept in small letters.
comments_require_approval	yes/no	needed	R-40.
submission_limit_count	number	1 or more	How many requests. Default 5.
submission_limit_window_minutes	number	1 or more	In how many minutes. Default 60.
features	free shape	needed	The feature switches. A missing switch uses the code default, so a broken row can never turn something on by mistake.
default_theme, default_language, default_sort	text	needed	The values used when a person changed nothing.
version	number	needed	Goes up by one on every change here or to the categories and statuses. This is how browsers know to get fresh settings (R-52).
updated_at, updated_by	date, id	FK → users	Who changed the settings last, and when.

12.4 categories

FIELD	KIND	RULES	WHAT IT IS
id	id	PK	
name	text	needed, UQ, 1–40	Bug, Feature, Improvement, Question — only as a start. The admin owns this list.
slug	text	needed, UQ	Short name for the web address and filters.
color	text	needed	Always shown with the text too, never colour alone (R-111).
description	text	can be empty	Small help text in the picker.
sort_order	number	needed	The order the admin chose.
is_active	yes/no	needed	No = retired (R-45).

12.5 statuses

FIELD	KIND	RULES	WHAT IT IS
id	id	PK	
name	text	needed, UQ, 1–40	New, Under Review, Planned, In Progress, Done, Declined — only as a start.
slug	text	needed, UQ	
color	text	needed	
sort_order	number	needed	The order of the pipeline the admin sees.
is_default	yes/no	only one row can be yes	The status every new request gets (R-11). The database makes sure there is only one.
is_terminal	yes/no	needed	Done, Declined. Used to make closed requests quieter (R-50).
is_active	yes/no	needed	Retired. The default one can never be retired (R-48).

12.6 feedback_requests

The main table. **The vote count and comment count are not here.** They are counted when we read, so they can never be wrong.

FIELD	KIND	RULES	WHAT IT IS
id	id	PK	
title	text	needed, 5–120	R-12.
description	text	needed, 10–5000	Plain text. Line breaks are kept. No formatting is read.

category_id	id	needed, FK, cannot be deleted	The database refuses to delete a used category (R-46).
status_id	id	needed, FK, cannot be deleted	Set by the server, changed only by an admin.
author_id	id	needed, FK, cannot be deleted	Because deleting an account wipes the name but keeps the content (R-61).
is_pinned, pinned_at	yes/no, date		Admin only. The date keeps pinned ones in a fixed order.
search_vector	search text	made by the database	Built from the title and the description, by the database itself, so it can never fall behind the real text.
created_at, updated_at	date	needed	updated_at changes only when the text changes — not on a vote or a comment.
last_activity_at	date	needed	Also moves when a comment arrives. This is what “last activity” sorting uses.

12.7 votes

FIELD	KIND	RULES	WHAT IT IS
id	id	PK	
request_id	id	needed, FK, goes with the request	Delete the request, the votes go too.
user_id	id	needed, FK, goes with the person	
created_at	date	needed	
request_id + user_id together must be unique. This one line is what makes R-26 true, even with ten clicks at the same second. Un-voting is deleting the row.			

12.8 comments

FIELD	KIND	RULES	WHAT IT IS
id	id	PK	
request_id	id	needed, FK, goes with the request	
author_id	id	needed, FK, cannot be deleted	Same reason as the request author.
body	text	1–2000; empty when deleted	Plain text.
state	text	published / pending / deleted	“pending” only when approval is on. “deleted” is the grey line (R-38).

created_at, updated_at, deleted_at	date		If updated is after created, we show “edited”.
moderated_by	id	FK → users	Which admin deleted, approved or rejected it.

12.9 request_status_history

FIELD	KIND	RULES	WHAT IT IS
id	id	PK	
request_id	id	needed, FK, goes with the request	
from_status_id	id	empty the first time	Empty when the request was created.
to_status_id	id	needed, FK	
changed_by	id	needed, FK → users	Which admin.
note	text	can be empty, max 500	Optional reason, shown to everyone.
created_at	date	needed	

12.10 invitations

Only used when the sign-up rule is invite-only.

FIELD	KIND	RULES	WHAT IT IS
id	id	PK	
email	text	needed, UQ	A name on a list. No invite email is sent (see Q-5).
invited_by	id	needed, FK → users	
accepted_at, accepted_user_id	date, id	can be empty	Filled in when that email signs in for the first time.

12.11 notification_outbox

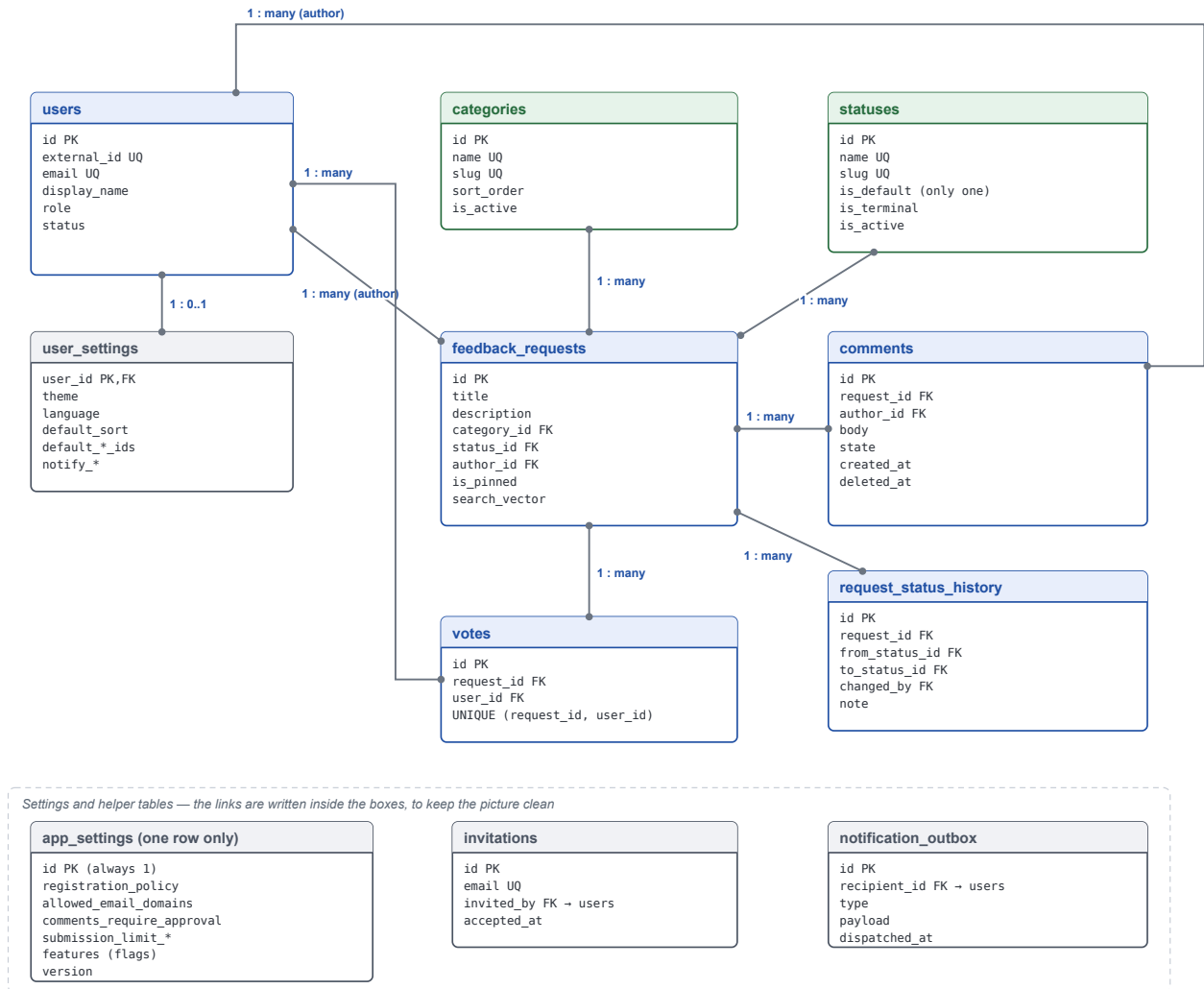
FIELD	KIND	RULES	WHAT IT IS
id	id	PK	
recipient_id	id	needed, FK → users	Who should get it.
type	text	from a fixed list	Comment on my request, or status change on my request.
payload	free shape	needed	The message text is saved now, so it stays right even if the request is edited later.
created_at, dispatched_at	date		In this version we only write it down and stamp it (R-73).

attempts, last_error	number, text		Here already, so real sending later is a small change.
----------------------	--------------	--	--

12.12 How the tables connect

LINK	HOW MANY	IF THE PARENT GOES	WHY
users → user_settings	1 to 0 or 1	goes too	Settings mean nothing without the person.
users → feedback_requests	1 to many	blocked	Deleting an account wipes the name, keeps the content (R-61).
users → comments	1 to many	blocked	Same.
users → votes	1 to many	goes too	A vote is a personal signal, not content.
users → request_status_history	1 to many	blocked	A history line must keep its admin.
categories → feedback_requests	1 to many	blocked	Retire, never delete (R-46).
statuses → feedback_requests	1 to many	blocked	Same.
feedback_requests → votes	1 to many	goes too	Votes belong to the request.
feedback_requests → comments	1 to many	goes too	Same.
feedback_requests → status history	1 to many	goes too	Same.
users → notification_outbox	1 to many	goes too	Messages for a gone account are dropped.
users → invitations	1 to many	blocked	Keeps who invited whom.

13. The ERD



Picture 5. All eleven tables. Blue = the main work. Green = the lists the admin owns. Grey = settings and helpers. Only the key fields are drawn; part 12 has all of them.

13.1 The same picture as Mermaid text

This is the source of the picture. Paste it into any Mermaid tool to see it drawn.

```
erDiagram
    USERS ||--o| USER_SETTINGS : "has settings"
    USERS ||--o{ FEEDBACK_REQUESTS : "writes"
    USERS ||--o{ COMMENTS : "writes"
    USERS ||--o{ VOTES : "gives"
    USERS ||--o{ REQUEST_STATUS_HISTORY : "changes status"
    USERS ||--o{ NOTIFICATION_OUTBOX : "gets message"
    USERS ||--o{ INVITATIONS : "invites"
    USERS ||--o| APP_SETTINGS : "last changed by"

    CATEGORIES ||--o{ FEEDBACK_REQUESTS : "groups"
    STATUSES ||--o{ FEEDBACK_REQUESTS : "marks"
    STATUSES ||--o{ REQUEST_STATUS_HISTORY : "from and to"
```

```

FEEDBACK_REQUESTS ||--o{ VOTES : "gets"
FEEDBACK_REQUESTS ||--o{ COMMENTS : "has"
FEEDBACK_REQUESTS ||--o{ REQUEST_STATUS_HISTORY : "keeps"

USERS {
  uuid      id          PK
  text      external_id  UK "id from the identity provider"
  text      email        UK
  boolean   email_verified
  text      display_name
  text      avatar_url    "empty means show initials"
  text      role          "user or admin"
  text      status        "active or deleted"
  timestamp created_at
  timestamp updated_at
  timestamp deleted_at
}

USER_SETTINGS {
  uuid      user_id      PK "points at users"
  text      theme        "empty means use the admin value"
  text      language
  text      default_sort
  uuid_list default_status_ids
  uuid_list default_category_ids
  boolean   notify_on_comment
  boolean   notify_on_status_change
  timestamp updated_at
}

APP_SETTINGS {
  int      id          PK "always 1"
  text     registration_policy "open, invite_only, domain_restricted"
  text_list allowed_email_domains
  boolean  comments_require_approval
  int      submission_limit_count
  int      submission_limit_window_minutes
  json     features      "feature switches"
  text     default_theme
  text     default_language
  text     default_sort
  int      version        "goes up on every change"
  timestamp updated_at
  uuid     updated_by      FK
}

CATEGORIES {
  uuid      id          PK
  text      name        UK
  text      slug        UK
  text      color
  text      description
  int      sort_order
  boolean   is_active    "false means retired"
  timestamp created_at
  timestamp updated_at
}

STATUSES {
  uuid      id          PK
  text      name        UK
  text      slug        UK
  text      color
  int      sort_order

```

```

        boolean    is_default      "only one row can be true"
        boolean    is_terminal    "Done, Declined"
        boolean    is_active
        timestamp   created_at
        timestamp   updated_at
    }

    FEEDBACK_REQUESTS {
        uuid        id              PK
        text        title           "5 to 120 letters"
        text        description     "10 to 5000 letters, plain text"
        uuid        category_id     FK "cannot be deleted"
        uuid        status_id       FK "cannot be deleted"
        uuid        author_id       FK "cannot be deleted"
        boolean     is_pinned
        timestamp   pinned_at
        tsvector    search_vector   "made by the database"
        timestamp   created_at
        timestamp   updated_at
        timestamp   last_activity_at
    }

    VOTES {
        uuid        id              PK
        uuid        request_id      FK "goes with the request"
        uuid        user_id         FK "goes with the person"
        timestamp   created_at
    }

    COMMENTS {
        uuid        id              PK
        uuid        request_id      FK "goes with the request"
        uuid        author_id       FK "cannot be deleted"
        text        body           "1 to 2000 letters, plain text"
        text        state          "published, pending, deleted"
        timestamp   created_at
        timestamp   updated_at
        timestamp   deleted_at
        uuid        moderated_by    FK
    }

    REQUEST_STATUS_HISTORY {
        uuid        id              PK
        uuid        request_id      FK "goes with the request"
        uuid        from_status_id  FK "empty the first time"
        uuid        to_status_id    FK
        uuid        changed_by      FK
        text        note
        timestamp   created_at
    }

    INVITATIONS {
        uuid        id              PK
        text        email           UK
        uuid        invited_by      FK
        timestamp   accepted_at
        uuid        accepted_user_id FK
        timestamp   created_at
    }

    NOTIFICATION_OUTBOX {
        uuid        id              PK
        uuid        recipient_id    FK "goes with the person"
        text        type

```

```
    json      payload
    timestamp  created_at
    timestamp  dispatched_at
    int        attempts
    text       last_error
}
```

Notice what is missing: there is no “vote count” field and no “comment count” field. They are counted from the real rows every time we read, so they can never be wrong. If one day that is too slow, we can add a counter later behind the same code — a small change, not a new design.

14. Rules that are always true

These must hold no matter which screen or which button caused it. Where the **database** is what stops it, we say so — a check in code and a check in the database are not the same strength when two things happen at the same second.

PROMISE	WHO STOPS IT	RULES
Only people the sign-up rule allows get an account.	Server	R-4, R-67
Only a checked email counts for the domain rule. An unchecked email is a claim, not a fact.	Server	R-67
Changing the sign-up rule never removes people who already got in.	Server	R-4
One person, one vote per request.	Database	R-26
The vote count always equals the real number of votes.	By design — there is no count field	R-28
Two categories or two statuses can never have the same name.	Database	R-44
Exactly one status is the first one.	Database	R-47
A category or status in use can never be deleted.	Database	R-46
There is always at least one category and one first status, so writing a request is always possible.	Server	R-48
A person cannot send more requests than the admin allows. Two at the same second cannot both slip through.	Server, inside one database step	R-69
Only the writer changes their own words. An admin deletes, never rewrites.	Server	R-13, R-35, R-36
Who owns a thing is read from the database, never from what the browser sent.	Server	R-7
A setting is picked as: code default, then admin, then user. Always in that order, in one place.	Server	R-51
A person can never change an admin setting.	Server	R-60
A feature switch that is off blocks the screen and the server.	Both	R-42

A missing or broken switch falls back to the code default, so a bad settings row can never switch something on by accident.	Server	R-42
Nobody is emailed about their own action, and only if they asked for it.	Server	R-71
An email problem can never undo a comment.	Server	R-72
Deleting an account keeps the writing and drops the name.	Server	R-61
The app can never be left with no admin.	Server	R-62

15. Every case on every screen

This is the list of cases we must build and test. The normal case is the easy one. The other four are where the marks are (hard part H-3).

15.1 Board

CASE	WHAT THE PERSON SEES
Normal	The list, with counts, pinned ones on top, page buttons, and the total number found.
Loading	Grey blocks the same size and shape as the rows. Nothing jumps when the real rows arrive.
Empty — no data	“No requests yet. Be the first.” With the button to write one. This is the first thing a new company sees, so it should not look broken.
Empty — filters	“Nothing matches these filters.” With a Clear filters button. This is a different message from the one above.
Error	“We could not load the board.” With a Try again button. Filters stay as they were.
Edge	Page 5 of a list that shrank to 2 pages → go back to the last real page, do not show an empty page. A search with no results → the filter message, not the error one. A category retired while filtering by it → keep working, show it as retired.

15.2 Request page

CASE	WHAT THE PERSON SEES
Normal	The request, the vote button in the right state, the comments, the comment box.
Loading	Grey blocks for the request and for the comments, loaded separately.
Empty	No comments yet → “No comments yet. Start the discussion.”
Error	Request failed to load → error with Try again. Comments failed but the request loaded → error only in the comments part; the rest of the page still works.
Not found	Deleted while it was open, or a bad address → “This request does not exist any more,” and a link back to the board.

Not allowed	Someone opens the edit page for a request that is not theirs → clear message, no form.
Edge	Comments switched off → the whole comment part is gone (R-42). Approval on → my own waiting comment is shown to me marked “waiting”. A deleted comment → a grey line, the thread keeps its shape. The author deleted their account → “Deleted user”.

15.3 New request / edit request

CASE	WHAT THE PERSON SEES
Normal	Three fields, a Save button, and a hint of similar requests while typing the title.
Bad input	A message under the field that says how to fix it. The cursor moves to the first bad field. Nothing is lost.
Over the limit	“You have sent 5 requests in the last hour. You can send another at 14:30.” The text they wrote is kept , not thrown away.
Error	Saving failed → the form stays filled in, with a message and a Try again button. Never a blank form.
Edge	The category was retired while the form was open → a clear message asking them to pick another. Two fast clicks on Save → only one request is made. Leaving with unsaved text → ask first.

15.4 Voting

CASE	WHAT HAPPENS
Normal	Click → the number goes up at once, the button shows “voted”.
Take it back	Click again → the number goes down, the button goes back.
Double click	Still one vote. No error message the person has to understand.
Error	The number goes back to what it was, and a small message says it did not save.
Edge	The request was deleted a second ago → a clear message and the person is sent back to the board. Voting on my own request → allowed.

15.5 Comments

CASE	WHAT HAPPENS
Normal	Write, save, it appears at the bottom without a reload, the box clears, the count goes up.
Waiting	Approval is on → only I see mine, marked “waiting for approval”. The count does not move.
Error	The text I typed is kept in the box, with a message and a Try again button.
Not allowed	Editing someone else's comment → refused by the server, even if the button was somehow visible.
Edge	Comments switched off while I am typing → a clear message when I press save, not a silent failure. A comment on a request that was just deleted → clear message.

15.6 My settings

CASE	WHAT HAPPENS
------	--------------

Normal	Each part saves on its own and says "Saved".
Bad input	Empty name → a message under the field.
Error	The old value comes back on screen, with a message. The person's typing is not lost.
Delete account	A question that says clearly: sign-in stops, posts stay as "Deleted user", votes go. They must type or click to confirm.
Edge	The last admin tries to delete their account → refused with the reason (R-62). A saved default filter points at a retired category → it is skipped, nothing breaks.

15.7 Admin screens

CASE	WHAT HAPPENS
Normal	Change a status, pin, delete a comment, add a category, change a setting. Everything shows at once.
Not allowed	A normal person types the admin address → the server refuses. The screen is not just hidden.
Blocked on purpose	Retire the first status → refused, with the reason. Retire the last category → refused. Delete a used category → refused, offer to retire instead.
Error	A setting failed to save → the old value stays on screen with a message. No half-saved settings.
Edge	Two admins change the same setting at once → the last one wins, and the version number goes up so other browsers pick it up. Turning off comments while people are typing → see 15.5.

15.8 Sign in and start-up

CASE	WHAT HAPPENS
Normal	Sign in, one call, the app appears.
Not allowed to join	The email does not pass the sign-up rule → a clear page saying why. Not a blank screen, not a loop back to sign-in.
Start-up call fails	An error page with a Try again button. Never a spinner forever, never a white page.
Token runs out while using the app	It is renewed quietly. If that fails, back to sign-in, and we remember which page they wanted.
Edge	The identity provider is down → a clear message saying sign-in is not available, not a broken page. An admin changed a setting a minute ago → the browser picks it up on its next refresh, not instantly (see Q-7).

16. Things that are not features

These are not buttons. They are how the whole app must behave.

16.1 Safety

#	RULE
R-93	Every check that decides “can this person do this” runs on the server , on the saved row. Hiding a button is only a kindness. Every “no” gets a test that proves the server says no.
R-94	Knowing an id gives nothing extra. Person A cannot reach person B's things by typing the id.
R-95	Everything sent to the server is checked against a written shape. Unknown fields are refused, not ignored. Every text and every number has a smallest and a biggest value.
R-96	Limits on the server: at most 100 rows per page, at most 1 MB per request, plus the admin's own limit on new requests. A rough limit per computer also guards every save.
R-97	Nothing a person types is ever put straight into a database query. Sort names and filter names come from a fixed list.
R-98	What people write is shown as plain text. We never read it as HTML and never as formatting. This removes a whole family of attacks in one line.
R-99	The API sends only the fields it means to. Another person's email or settings are never sent. On a request and a comment we send only the writer's name and picture.
R-100	An error message never shows the inside of the system: no stack, no database text, no library names. The full story goes to the log, with an id that is also given to the person so support can find it.
R-101	Normal safety headers are set. Only our own web addresses are allowed to call the API.
R-102	No secret is ever put in the repository. Only an example file with fake values. Containers do not run as root.

16.2 Speed

For the size in part 1, on a normal development machine, not counting the first cold start.

#	RULE
R-103	The board (search + filter + sort + page + counts): under about 300 ms for almost every call.
R-104	A request page, and saving a vote or a comment: under about 250 ms.
R-105	The start-up call: one call, under about 200 ms. This is a promise about the shape, not only the speed.
R-106	The database has an index for every filter and every sort we offer, and for the search. No core screen reads the whole table.
R-107	The first paint is quick, and nothing on the page jumps when the data lands.

16.3 Everyone can use it

#	RULE
---	------

R-108	We aim at the normal accessibility standard (WCAG 2.2 AA). Anything we know we did not reach goes in the README instead of being left to be found.
R-109	Everything works with the keyboard only. You can always see where you are. A skip-to-content link at the top.
R-110	Every control has a name a screen reader can say. A control with a state (the vote button, a filter) says its state too, not only its name.
R-111	Colour is never the only way to tell something. A status and a category always show their text next to their colour. Text has enough contrast in both light and dark.
R-112	Things that finish in the background are read out. When a form fails, the cursor goes to the first bad field.
R-113	The page still works at 200% zoom and down to a 320 pixel wide screen, with no sideways scrolling.
R-114	If the computer asks for less movement, we use less movement.

16.4 Running it and keeping it right

#	RULE
R-115	Important promises are kept by the database , not only by code: one vote per person, links between tables, only one first status.
R-116	Side jobs that are not part of what the person asked for (emails) can never break what they asked for.
R-117	The same action done twice at the same moment ends in one result, not an error and not a double.
R-118	The API keeps nothing in its own memory between calls, so any copy of it can answer any call.
R-119	Logs are machine readable, with an id that follows one call through the system. Never a password, a token, or a person's private data in a log.
R-120	The example data can be loaded again and again without breaking, and it is rich enough to see every case in part 15 — including the empty ones.

16.5 Keeping the code readable

#	RULE
R-121	One way to do each thing: errors, checks, pages, permissions, loading states. Decided once, then reused. This is hard part H-9 and the brief names consistency directly.
R-122	Strong types everywhere. No untyped holes at the edges of the app.
R-123	Each part of the code owns its own tables. Other parts ask it, they do not touch its tables.
R-124	Tests must cover at least: every permission rule with its no case, the input limits, how a setting is picked, voting twice, the request limit, and the board's search, filter, sort and pages.

17. Tools we must choose

The brief asks for some of these by name, and asks us to explain the choice. **Nothing is chosen yet.** Each choice, with its reasons, goes in DECISIONS.md when it is made.

WHAT	WHAT THE BRIEF SAYS	WHAT WE MUST STILL DECIDE AND WRITE DOWN
------	---------------------	--

Front end	Angular. Explain the version.	Which version, and why. How we hold the state on screen. How we style it. How we build the components.
Back end	Node.js is preferred. Anything else must be explained in writing.	Node or not. If not, the written reason.
Sign-in	An open source identity system. Email and password, plus at least one social login. Do not build sign-in yourself.	Which one, and why. How it is set up so it starts with the app and has example users.
Database	Your choice. Explain it.	Which one, and why it fits this data.
Shape of the system	Several services are preferred, but the reasoning matters more than the shape . A good reason for something simple beats splitting it up for no reason.	How many pieces, where the lines are, and the argument for it.
Running it	Containers, settings from the environment, ready for an orchestrator. A small local Kubernetes is fine.	Which local Kubernetes, and how the files are laid out.
Tests	The README must say how to run them.	Which tools, and which layers we test.
API document	Optional.	Whether we produce one. (It is cheap, so probably yes.)

The brief warns about one of these twice. Splitting the app into many services just to look serious will **lose** marks if the reason is weak. A simple shape with a good written argument will **gain** them.

18. How we know a feature is done

Written as: **given** a starting point, **when** someone does something, **then** this must be true. These are the sentences the tests check.

Sign in

- Given a new person and the open sign-up rule, when they sign in with email and password or with a social account, then an account is made and they land on the board with their name in the corner.
- Given the domain rule allows `example.com`, when someone with a checked `@other.com` email signs in, then they see a clear message and **no account is made**.
- Given the domain rule, when someone with an **unchecked** `@example.com` email signs in, then they are refused, and the message says email checking is the reason.
- Given nobody is signed in, when the API is called directly, then it answers 401 — not a redirect, and not data.

Write a request

- Given a signed-in person, when they save a good title, text and category, then the request exists with the first status, their name, zero votes, and it is on the board without a manual refresh.
- Given a four letter title, when they press save, then a message under the field says the smallest length, the cursor goes there, and nothing is saved.
- Given a browser that also sends a status or a vote count, when the server reads it, then those extra fields are refused and the server's own values are used.
- Given a title like one that already exists, when they stop typing, then up to five similar requests are shown, and they can still save theirs.

Use the board

- Given a board with data, when a word from a description is searched, then the matching requests come back and the search is in the web address.
- Given two statuses and one category are ticked, when the list loads, then only requests that match (status A or status B) **and** that category are shown, with the total.
- Given filters that match nothing, when the list loads, then the “nothing matches these filters” message appears with a Clear button — not the “no requests yet” message.
- Given a pinned request, when a filter hides it, then it is **not** shown; when a filter includes it, then it is first.
- Given a browser asking for 10000 rows per page, or a made-up sort name, then the server gives at most 100 rows and refuses the sort name.

Vote

- Given a request I have not voted for, when I use the button with the mouse **or** the keyboard, then the count goes up by one and the button shows I voted.
- Given I already voted, when I use it again, then my vote is taken back and the count goes down by one.
- Given ten votes sent at the same moment by the same person, when they are all handled, then exactly one vote exists and no call fails with a confusing error.

- Given the server refuses, when the screen puts the number back, then the old number is shown with a short message.

Comment

- Given a signed-in person, when they save a comment, then it appears without a reload, the box clears, and the count goes up.
- Given someone else's comment, when a normal person tries to edit it through the API, then the server answers 403 and nothing changes.
- Given a comment in the middle, when its writer deletes it, then a grey line takes its place, the order stays, and the count goes down.
- Given approval is on, when a person saves a comment, then only they and admins see it, marked as waiting, and it is not counted until an admin approves it.

My own things

- Given my own request, when I change the text, then it is saved and shown as edited.
- Given someone else's request, when a normal person calls edit or delete with its id, then the server answers 403 and nothing changes — whatever the screen showed.
- Given my own request with votes and comments, when I delete it after confirming, then it and its votes and its comments are gone from the board.

Admin work

- Given an admin, when they change a status, then the new status shows everywhere the request appears, and who did it and when is saved.
- Given an admin, when they pin a request, then it is first for everyone, still following the filter rule above.
- Given a normal person, when they call any admin action, then the server answers 403 — whether or not the screen ever showed it to them.
- Given an admin, when they delete a rude comment, then it becomes a grey line and the admin's name is saved against it.

The feature switch

- Given comments are on, when I open a request, then the thread and the box are there and counts show on the board.
- Given an admin turns comments off, when I next load the app, then the box and the thread are gone and counts are gone everywhere.
- Given comments are off, when a browser sends a comment straight to the API, then the server answers 403 with a message saying comments are switched off. **This is the test that proves the switch is real.**

Categories and statuses

- Given an admin adds a category, when people next load the app, then they can choose it.
- Given a category used by old requests, when the admin retires it, then it is gone from the picker and those old requests still show it correctly.
- Given the first status, when an admin tries to retire it, then it is refused with the reason.

- Given one status is the first one, when the admin makes another one first, then exactly one first status exists afterwards.

Settings and start-up

- Given a fresh load, when we watch the network, then there is **exactly one** settings call before the app shows — not a chain.
- Given my theme is dark and the admin default is light, when I load the app, then it is dark, with no white flash first.
- Given I never set a sort, when the admin changes the default sort, then my board uses the new one — and someone who did set their own keeps theirs.
- Given a browser trying to change an admin setting through the user API, then it is refused and nothing is saved.
- Given the start-up call fails, when the app starts, then an error with a Try again button is shown — not a white page and not an endless spinner.

The request limit

- Given a limit of 3 per hour and a person who sent 3, when they send a fourth, then it is refused, the message says when they can send again, and nothing is saved.
- Given the same person an hour later, when they send one, then it works.
- Given the admin raises the limit, when the person tries again, then it works — with no restart.

Profile and account

- Given I change my name, then it changes on my old requests and comments too.
- Given my theme follows the computer, when the computer switches to dark, then the app switches too, with no reload.
- Given I change the language, then the screen changes at once, and any missing word shows in English, not as a code word.
- Given I delete my account after a question that said my posts will stay, then my sign-in stops working, my posts show “Deleted user”, and my votes are gone.
- Given I am the only admin, when I try to delete my account, then it is refused with the reason.

Running it

- Given a fresh copy of the repository on a machine with only Docker, when the written command is run, then the front end, API, database and identity provider all start, the database is set up, example data is loaded, and a written test account can sign in — with no secret extra step.
- Given the Kubernetes files, when they are applied to a small local cluster, then the database step finishes **before** the app is marked ready, and the health checks tell the truth.
- Given the repository, when it is searched for secrets, then there are none — only example values.

Keyboard and small screens

- Given only a keyboard, when I go through the board, open a request, vote, comment and open a pop-up, then every step works, I can always see where I am, and the pop-up gives the keyboard back when it closes.

- Given a 360 pixel wide screen, when I open any page, then it fits with no sideways scrolling and every control still works.
- Given a form that fails, when I press save, then the problem is read out, the cursor goes to the first bad field, and each message says how to fix it.

19. What is not clear in the brief

The brief says: “If anything in this brief is unclear, ask us. Sending questions is not a mark against you.” These are the questions. Until they are answered, the right column is what we build. The ones marked Δ would change real work if the answer is different.

PROBLEMS IN THE DOCUMENT ITSELF

#	WHAT WE NOTICED	WHAT WE DO UNTIL WE KNOW
Q-1	Δ There is no Section 2. The brief goes from part 1 straight to part 3.	We assume it had no rules in it. But if it had a deadline or a time limit , that changes what we build more than anything else here. This is the question we most want answered.
Q-2	Δ Part 7 ends with “Two things worth stating plainly:” and then nothing. Two sentences seem to be missing from the file.	We work without them and say so here, instead of guessing.
Q-3	Part 6 has no 6.6 (it goes 6.5 then 6.7).	Looks like a numbering slip. We use the headings that exist.
Q-4	Part 4.3 ends with the half sentence “Clearly defined requirements.”	We read it as: the README must clearly say what is needed to run the app. That is what we will write.
Q-5	Part 4.1 says “an open-source identity, Support email/password sign-in”.	We read it as “an open-source identity provider ”, and we read “open source” strictly: something we can run ourselves from a container in this repository, not a paid service with a free plan.

QUESTIONS ABOUT THE PRODUCT

#	QUESTION	WHAT WE DO UNTIL WE KNOW
Q-6	Δ Can someone browse the board without signing in? Part 3 says “everyone can browse”, but every journey in part 3.4 starts with signing in.	Sign-in is needed for everything. It is an internal board, so this is the safer choice. We write the read side so that opening it up later is a one line change.
Q-7	Δ Where is the sign-up rule enforced, when the identity provider owns the sign-up screen?	In our app, the first time the person signs in. We also set the identity provider's own switch to match. One visible result: a person can have an account at the identity provider and still not get into the app.
Q-8	“Invite only” needs an invite system, and the brief never describes one.	The small version: an admin adds an email to a list and sees if it was used. No invite email is sent.
Q-9	For the domain rule, can we trust the email a social account gives us?	Only if the identity provider says it is checked. An unchecked email is refused with a message that says why.
Q-10	How does the first admin exist?	It comes with the example data, both in the identity provider and in our database. The test password is written in the README and is for local use only.
Q-11	When comment approval is on, who can see a waiting comment?	The person who wrote it (marked “waiting”) and admins. Nobody else. It is not counted.

Q-12	Can many requests be pinned? Does pinning beat the sort the person chose?	Many can be pinned. Pinned ones come first inside the filter the person chose , then the normal sort. Pinning must never break a filter.
Q-13	Can a person vote for their own request? Does a new request start with one vote?	Yes they can vote for their own. No auto vote — zero is more honest than a fake one.
Q-14	Can an admin delete a category or status, or only retire it?	Only retire. Deleting one that is used would either break old requests or quietly change them.
Q-15	Does a new request need a starting status?	Yes. One status is marked as the first one. People never pick a status; only admins change it.
Q-16	What does search look at?	The title and the description. Not the comments — mixing them makes the result list confusing.
Q-17	What is the unit of the request limit?	N requests in M minutes, both set by the admin. We start at 5 in 60 minutes.
Q-18	How many languages must the language setting really support?	The mechanism, shown with two languages. Translating every word is a writing job, not a building job. Missing words fall back to English on screen.
Q-19	⚠ What should deleting an account do to what the person wrote?	Keep the writing, drop the name. If a full legal erase is meant, that is a different feature and we need to be told.
Q-20	Must the Kubernetes files really be running, or is writing them enough?	We will do both: write them to be read, and really run them on a small local cluster. If we cannot finish, the README will say which parts were never run.
Q-21	Is there a target for how much of the code tests must cover?	No number is given, so we test the things it would be embarrassing to get wrong (part 16.5, R-124) instead of chasing a percentage.
Q-22	How fast must an admin's change reach other people's browsers?	On their next settings refresh, not the same second. No live push. We say this out loud instead of pretending it is instant.

20. Where each rule comes from

PART OF THE BRIEF	RULES HERE
3.1 The main thing (a request)	R-10 ... R-15, R-28, R-43, part 12.6
3.2 Votes and comments	R-26 ... R-31, R-32 ... R-42, parts 12.7 and 12.8
3.3 The two roles	Part 4, R-8, R-63 ... R-70
3.4 The journeys	Part 6 (every one, one by one)
3.5 Settings	R-51 ... R-62, R-67 ... R-70, R-43 ... R-50, R-71 ... R-74
3.5 “where configuration lives ... without a chain of blocking requests”	R-51, R-52, R-53, R-105, pictures 3 and 4 — hard part H-4
3.5 “at least one feature flag”	R-42, part 11 — hard part H-5

4.1 The stack	R-1 ... R-5, R-75, R-79, part 17
4.2 The shape of the system	R-123, part 17
4.3 Running it	R-80 ... R-84, R-120 — hard part H-8
5 What we hand in	R-78, plus README, DECISIONS, SCOPE and the AI write-up
6 Working with AI	The AI write-up — hard part H-1
7 “correctness and security”	R-93 ... R-102, part 14 — hard part H-2
7 “user experience”	R-85 ... R-92, R-108 ... R-114, part 15 — hard part H-3
7 “engineering quality”	R-121 ... R-124 — hard part H-9
8 Asking questions	Part 19

Remember what this file is. It says what the app **must** do. It does not say what is built, because nothing is built yet. When we start, README.md is the place that tells the truth about what works. If we drop a rule from this file, the reason goes in SCOPE.md — we do not quietly delete it here.